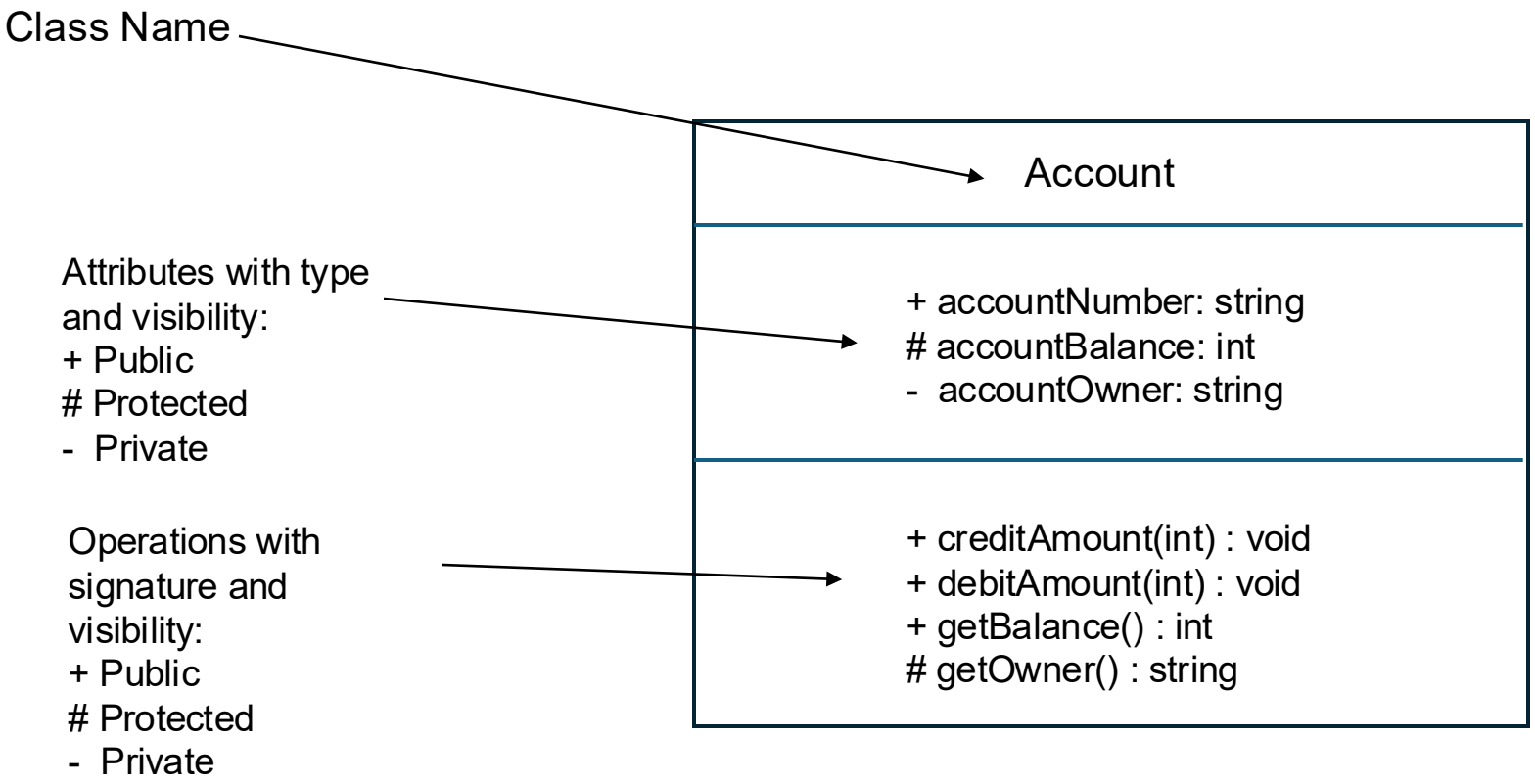
UML Class Diagrams

Unified Modeling Language (UML)

- UML is a general-purpose modeling language
- Standardized visual method of communicating Software Designs
- We will focus on a small subset of UML
    - Class diagrams and relationships

Class Diagrams



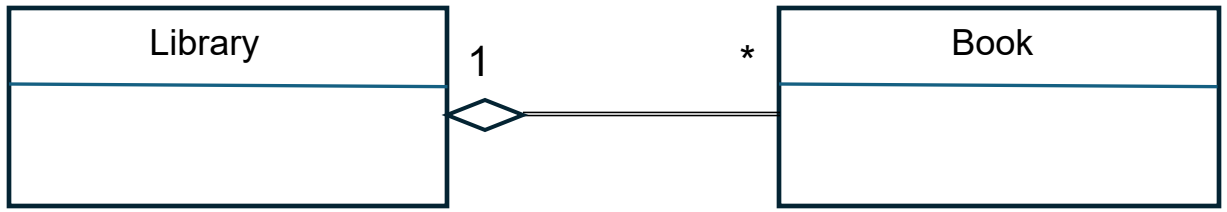
Relationships

- Association
- Order has a customer (multiple [*] orders may have the same [1] customer)
- Direction of arrow shows which entity applies the verb
- Association can be bidirectional (customer has orders, an order has a customer) --> arrow does not have arrow-head then



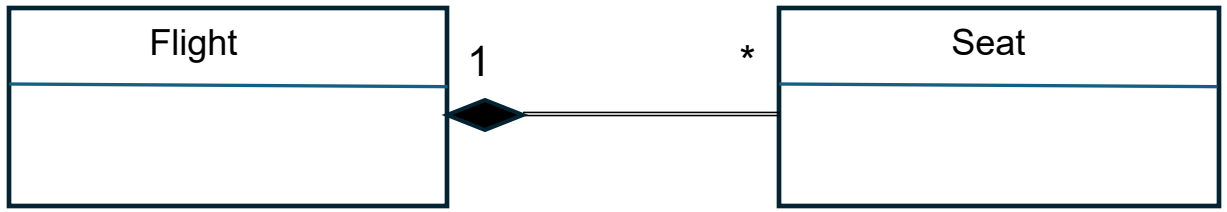
Relationships

- Aggregation
- Library has books (A[1] library has one or more [*] books)



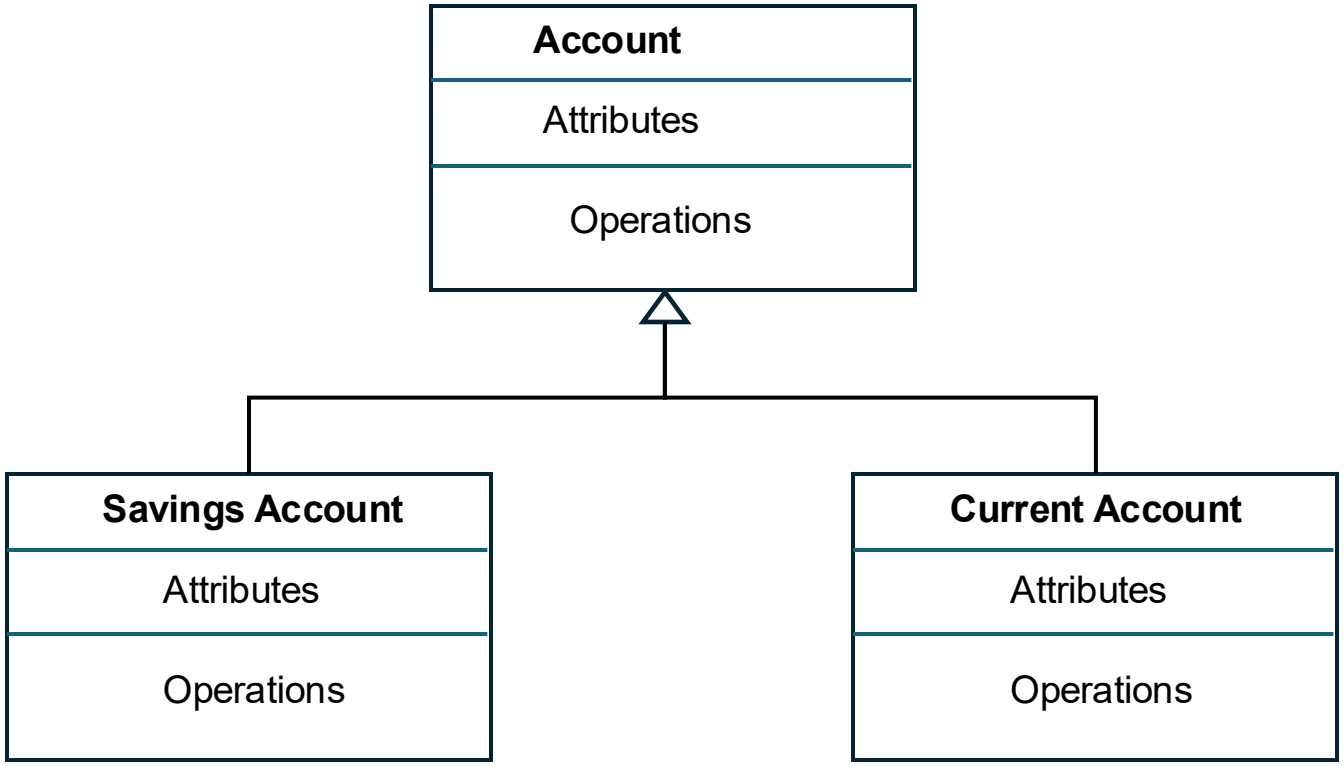
Relationships

- Composition
- Flight has seats (A flight [1] has many [*] seats)

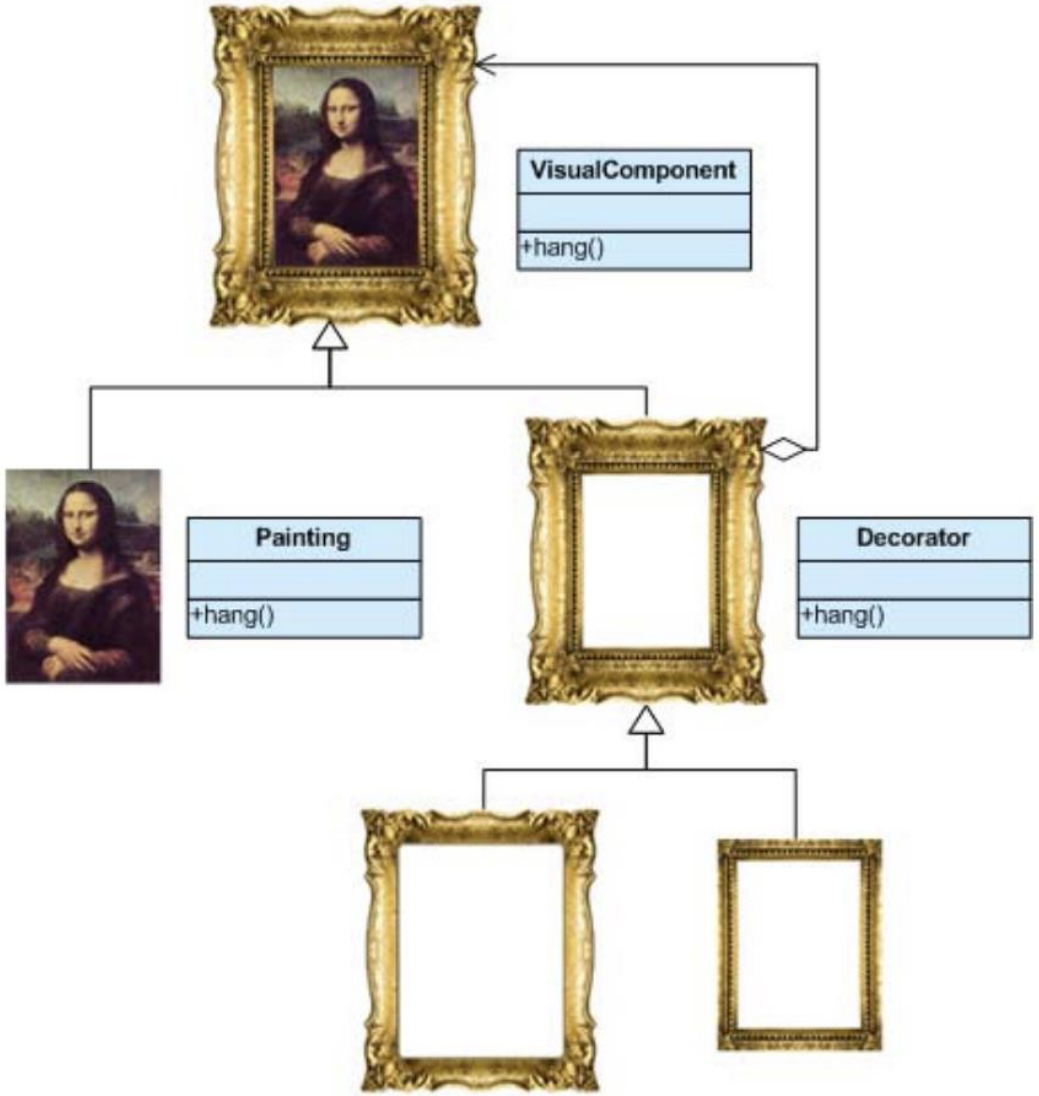


Relationships

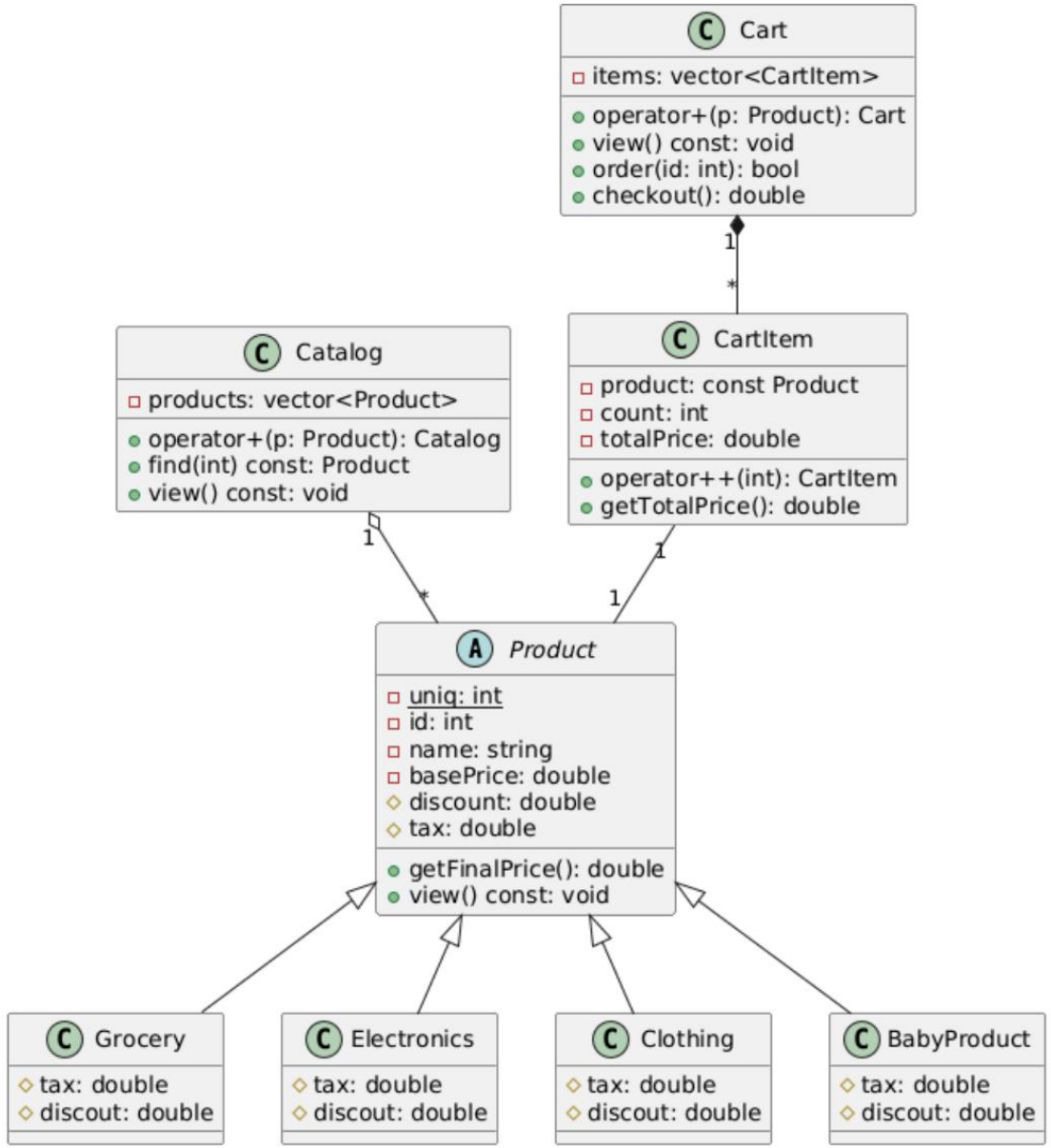
- Generalization (Inheritance)



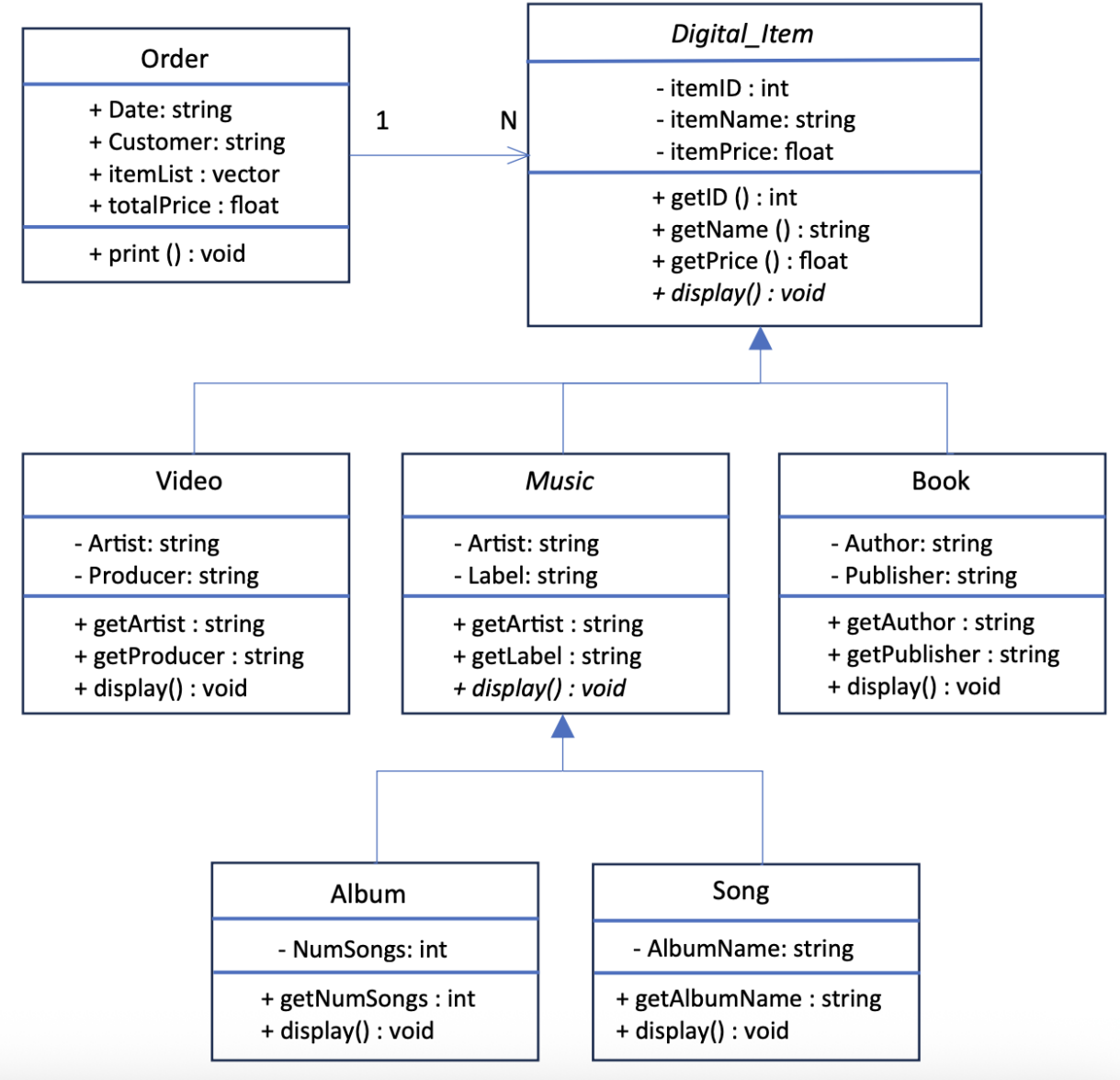
Building Object Hierarchies



Samples from Previous Exams



Samples from Previous Exams



Smart Pointers

Characteristics

- Templated pointer classes
- Manages memory on their own (no need for explicit new/delete)
- #include<memory>

Unique pointer

```
#include <iostream>
#include <memory>
using namespace std;
class Rectangle {
int length, breadth;
public:
      Rectangle(int l, int b) : length(l), breadth(b) {}
int area() {
       return length * breadth;
  }
};
int main() {
     unique_ptr<Rectangle> ptr1 = make_unique<Rectangle>(10, 5);
     cout << ptr1->area() << endl;
     unique_ptr<Rectangle> ptr2 = move(ptr1);
     cout << ptr2->area();
     return 0;
}
```

- Only one unique_ptr can own an object at a time.
- Lightweight and efficient.
- Ideal for single ownership scenarios.
- make_unique() to create a unique_ptr.
- Cannot be copied.
- Use ***move() constructor*** to change ownership

Shared pointer

```
#include <iostream>
#include <memory>
using namespace std;
class Rectangle {
    int length, breadth;
public:
    Rectangle(int l, int b) : length(l), breadth(b) {}
    int area() {
        return length * breadth;
    }
};
int main() {
    shared_ptr<Rectangle> ptr1 = make_shared<Rectangle>(10, 5);
    shared_ptr<Rectangle> ptr2 = ptr1;
    cout << ptr1->area() << endl;
    cout << ptr2->area() << endl;
    cout << ptr1.use_count();
    return 0;
}
```

- Supports shared ownership.
- Uses reference counting to manage the object's lifetime.
- Multiple shared_ptr objects can own the same object.
- make_shared() to create a shared_ptr.
- Directly copy (copy-constructor)

Weak pointer

```
#include <iostream>
  #include <memory>
  using namespace std;
  class Rectangle {
      int length, breadth;
  public:
      Rectangle(int l, int b) : length(l), breadth(b) {}
      int area() {
          return length * breadth;
      }
  };
  int main() {
      shared_ptr<Rectangle> ptr1 = make_shared<Rectangle>(10, 5);
      weak_ptr<Rectangle> ptr2 = ptr1;
      cout << ptr1->area() << endl;
      cout << ptr2.use_count();
      return 0;
  }
```

- Does not own the managed object.
- Does not increase the shared_ptr reference count.
- Helps prevent circular ownership.
- Uses lock() to obtain a temporary shared_ptr when the object is still alive.

Why C++

- *"Ferrari of programming languages" --> high-maintenance, high speed*
- Adobe systems (Photoshop, Illustrator,..)
- Microsoft Office Suite
- Databases: MySQL, Oracle
- Browsers: Google Chrome, Mozilla firefox
- ...
- ... massive scale with high performance

Thank you

https://www.youtube.com/watch?v=lI7tMxzSJ7w (thanks to Prof. Sujit)

